

TypeScript You MIGHT Like

Author: Bob Fornal

Entrepreneur

Code-squid provides solid, in-depth frontend training that is supported with real-world code projects.

Blessed husband and proud father of two.

Microsoft MVP | Torc Ambassador |

Senior Solutions Developer II at Leading EDJE, Inc.

Passionate about learning, testing, mentoring, speaking, and personal growth.

I've had quite a few conversations over the years where the developers are polarized. About half LOVE TypeScript and the other half HATE it. The ones who hate it are usually fighting against the type system, writing tons of type annotations, and getting frustrated with the error messages. In this talk, I want to share some patterns and best practices that have helped me and my teams use TypeScript in a way that is actually enjoyable and beneficial. My goal is to show that TypeScript doesn't have to be painful, and that with the right approach, you MIGHT actually ... like it.

TypeScript: We've All Been There

```
16 type SpiralOfDeath<  
17   N extends number,  
18   Acc extends unknown[] = []  
19   > = Acc["length"] extends N  
20     ? Acc  
21     : SpiralOfDeath<N, [WatchTheVideo<Day7>, ...Acc]>;  
22  
23   (parameter) victim: {  
24     crawlThroughTV: {  
25       crawlThroughTV: {  
26         crawlThroughTV: {  
27           crawlThroughTV: {  
28             crawlThroughTV: {  
29               crawlThroughTV: {  
30                 crawlThroughTV: {  
31                   crawlThroughTV: {  
32                     crawlThroughTV: {  
33                       crawlThroughTV: {  
34                         crawlThroughTV: ...; }  
35                     };  
36                 };  
37             };  
38         };  
39     };  
40 }  
  
victim: Samara<SpiralOfDeath<50>>  
}; WatchTheVideo<Day7> {  
  // @ts-expect-error - The curse cannot be satisfied  
  return victim.crawlThroughTV.wellOfSorrow.beforeYouDie.sevenDays;  
}
```



The ongoing Horror Show ...

- Async/Await Issues (Freddy Krueger's Promise Nightmare).
- Circular Type References (The Hills Have Eyes: inbred mutations).
- Conditional Types Gone Wrong (The Thing - shapeshifting horror).
- Deeply Nested Object Access (Hereditary's ancestral curse).
- Generic Type Inference Hell (Hellraiser's Cenobites).
- Massive Instances (Godzilla - IDE destroyer).
- Method Chaining with Complex Types (The Ring's 7-day curse).
- Union Type Narrowing Failure (Get Out - the Sunken Place).

horror-show.ts

```
// The Ring - Cursed Type System  
// "You will die in 7 days... or your type checker will"  
  
type Day1 = { cursed: true };  
type Day2 = { cursed: Day1 };  
type Day3 = { cursed: Day2 };  
type Day4 = { cursed: Day3 };  
type Day5 = { cursed: Day4 };  
type Day6 = { cursed: Day5 };  
type Day7 = { cursed: Day6 };  
  
type WatchTheVideo<T> = T extends Day7  
  ? "You have watched the cursed video. Your type system will now suffer eternal recursion."  
  : never;  
  
type SpiralOfDeath<  
  N extends number,  
  Acc extends unknown[] = []  
> = Acc["length"] extends N  
  ? Acc  
  : SpiralOfDeath<N, [WatchTheVideo<Day7>, ...Acc]>;
```

```

    : SpiralOfDeath<N, [WatchTheVideo<Day7>, ...Acc]>;

type Samara<T> = T extends any
  ? {
    crawlThroughTV: Samara<{
      wellOfSorrow: T;
      beforeYouDie: Samara<T>;
      sevenDays: Samara<Samara<Samara<T>>>>;
    }>;
  }
  : never;

// The cursed function - hover over the parameter type to see the horror
function watchCursedVideo(
  // Hover over 'victim' to witness the suffering
  victim: Samara<SpiralOfDeath<50>>
): WatchTheVideo<Day7> {
  // @ts-expect-error - The curse cannot be satisfied
  return victim.crawlThroughTV.wellOfSorrow.beforeYouDie.sevenDays;
}

// Attempt to call the cursed function
const cursedTape = {} as Samara<SpiralOfDeath<50>>;

// Hover over this to see your doom
watchCursedVideo(cursedTape);

// Bonus horror: Infinite type instantiation
type TheWell<T = TheWell> = {
  deeper: TheWell<TheWell<T>>;
  darker: T extends TheWell ? TheWell<T> : never;
  drowned: TheWell;
};

// Hover over this for maximum suffering
type SamarasRevenge = TheWell extends { deeper: infer D }
  ? D extends { darker: infer K }
    ? K extends { drowned: infer S }
      ? TheWell<TheWell<TheWell<S>>>
        : never
      : never
    : never;

// The final curse - uncomment at your own risk
// type InstantDeath = SamarasRevenge["deeper"]["darker"]["drowned"]["deeper"]["darker"];

```

async-await-type-inference-issues.js

```
// Painful code - vague promise-related errors
async function fetchUserData(id: string) {
  const response = await fetch(`/api/users/${id}`);
  return response.json();
}

async function processUser(id: string) {
  const user = fetchUserData(id); // Forgot await!

  // Error: Property 'name' does not exist on type 'Promise<any>'
  // The error doesn't say "Did you forget await?"
  console.log(user.name);

  // Later in the code...
  // Error: This expression is not callable.
  // Type 'Promise<any>' has no call signatures.
  user.toString();
}
```

circular-type-references.js

```
// Painful code - "Type instantiation is excessively deep and possibly infinite"
interface Node {
  value: string;
  children?: Node[];
  parent?: Node;
  metadata?: {
    related?: Node[];
    history?: Array<{
      node: Node;
      timestamp: number;
    }>;
  };
};

function findNode(root: Node, predicate: (node: Node) => boolean): Node | undefined {
  // Error: Type instantiation is excessively deep and possibly infinite
  // No indication of HOW to fix it
  if (predicate(root)) return root;

  return root.children?.find(child => findNode(child, predicate));
}
```

conditional-types-gone-wrong.js

```
// Painful code - error messages are incomprehensible
type ExtractPromiseType<T> = T extends Promise<infer U>
  ? U extends Array<infer V>
    ? V extends { id: infer ID }
      ? ID
      : never
    : never
  : never;

type Result = ExtractPromiseType<Promise<Array<{ id: string; name: string }>>>>;

// Trying to use it produces nightmares
function processResult<T>(data: T): ExtractPromiseType<T> {
  // Error: Type 'T' does not satisfy the constraint 'Promise<{ id: unknown; }[]>'
  // Type 'T' is not assignable to type 'Promise<{ id: unknown; }[]>'
  // ... 15 more lines of error message
  return data as any;
}
```

deeply-nested-object-access.js

```
// Painful code - vague "possibly undefined" errors everywhere
interface ApiResponse {
  data?: {
    user?: {
      profile?: {
        settings?: {
          notifications?: {
            email?: boolean;
          };
        };
      };
    };
  };
}

function getEmailNotificationSetting(response: ApiResponse) {
  // Error: Object is possibly 'undefined'
  // But which object? Good luck finding it!
  return response.data.user.profile.settings.notifications.email;
}

// Attempting to fix with optional chaining still gives unclear errors
function saveSettings(response: ApiResponse) {
  const email = response.data?.user?.profile?.settings?.notifications?.email;

  // Error: Argument of type 'boolean | undefined' is not assignable to parameter of type
  // 'boolean'
  // Where did the undefined come from? Which level?
  updateEmailPreference(email);
}

function updateEmailPreference(enabled: boolean) {
  console.log(enabled);
}
```

generic-type-inference-hell.js

```
// Painful code - generic error messages that don't help
function processData<T extends { id: string }>(  
  items: T[],  
  transformer: <U>(item: T) => U,  
  validator: <V>(transformed: U) => V  
) {  
  // Error: 'U' only refers to a type, but is being used as a value here  
  // Wait, what? How do I fix this?  
  return items.map(item => validator(transformer(item)));  
}  
  
// Usage that produces cryptic errors  
const result = processData(  
  [{ id: '1', name: 'test' }],  
  item => ({ ...item, processed: true }),  
  transformed => transformed.name.toUpperCase()  
);
```

long-instance.js

```
export interface CardObject {
  id: string;
  name: string;
  supertype: string;
  subtypes: Array<string>;
  level: string;
  hp: string;
  types: Array<string>;
  evolvesFrom: string;
  evolvesTo: string;
  rules: Array<string>;
  ancientTrait: {
    name: string;
    text: string;
  };
  abilities: Array<{
    name: string;
    text: string;
    type: string;
  }>;
  attacks: Array<{
    cost: Array<string>;
    name: string;
    text: string;
    damage: string;
    convertedEnergyCost: number;
  }>;
  weaknesses: Array<{
    type: string;
    value: string;
  }>;
  resistances: Array<{
    type: string;
    value: string;
  }>;
  retreatCost: Array<string>;
  convertedRetreatCost: number;
  set: Array<SetObject>;
  number: string;
  artist: string;
  rarity: string;
  flavorText: string;
  nationalPokedexNumbers: Array<number>;
  legalities: Array<{
    standard: string;
    expanded: string;
    unlimited: string;
```

```

    }>;
    regulationMark: string;
    images: {
        small: string;
        large: string;
    };
    tcgplayer: Array<{
        url: string;
        updatedAt: string;
        prices: Array<{
            low: number;
            mid: number;
            high: number;
            market: number;
            directLow: number;
        }>;
    }>;
    }>;
    cardmarket: Array<{
        url: string;
        updatedAt: string;
        prices: Array<{
            averageSellPrices: number;
            lowPrice: number;
            trendPrice: number;
            germanProLow: number;
            suggestPrice: number;
            reverseHoloSell: number;
            reverseHoloLow: number;
            reverseHoloTrend: number;
            lowProceExPlus: number;
            avg1: number;
            avg7: number;
            avg30: number;
            reverseHoloAvg1: number;
            reverseHoloAvg7: number;
            reverseHoloAvg30: number;
        }>;
    }>;
}

```

```

export interface SetObject {
    id: string;
    name: string;
    series: string;
    printedTotal: number;
    total: number;
    legalities: Array<{
        standard: string;
        expanded: string;
    }>;
}

```

```

    unlimited: string;
  }>;
  ptcgoCode: string;
  releaseDate: string;
  updatedAt: string;
  images: Array<{
    symbol: string;
    logo: string;
  }>;
}

```

method-chaining-with-complex-types.js

```

// Painful code - error appears far from the actual problem
class QueryBuilder<T> {
  private conditions: any[] = [];

  where<K extends keyof T>(field: K, value: T[K]) {
    this.conditions.push({ field, value });
    return this;
  }

  execute(): Promise<T[]> {
    return Promise.resolve([] as T[]);
  }
}

interface User {
  id: number;
  name: string;
  email: string;
}

// Error appears at the end of the chain, not where the mistake is
const query = new QueryBuilder<User>()
  .where('name', 'John')
  .where('email', 'john@example.com')
  .where('age', 25) // Typo: 'age' doesn't exist, but error is vague
  .where('id', 1)
  .execute(); // Error: shows up here with confusing message about the entire chain

```

union-type-narrowing-failure.js

```
// Painful code - TypeScript can't narrow the type properly
type SuccessResponse = {
  status: 'success';
  data: { value: number };
};

type ErrorResponse = {
  status: 'error';
  error: { message: string };
};

type Response = SuccessResponse | ErrorResponse;

function handleResponse(response: Response) {
  if (response.status === 'success') {
    // Error: Property 'data' does not exist on type 'Response'
    // Property 'data' does not exist on type 'ErrorResponse'
    // But we just checked the status!
    console.log(response.data.value);
  }
}

// Even worse with multiple conditions
function complexHandling(response: Response, shouldLog: boolean) {
  if (shouldLog) {
    console.log('Processing...');
  }

  if (response.status === 'success') {
    // STILL doesn't narrow properly after unrelated condition
    console.log(response.data.value); // Error!
  }
}
```

"TypeScript error message from hell"

"We've all been here. Let's fix that."

Key Message "TypeScript is a tool, not a religion. Use what helps, skip what doesn't."

What We'll Cover

1. Just Enough Types Philosophy.
2. Patterns That Prevent Bugs.
3. Features You're Probably Overlooking.
4. Keeping Types Manageable.
5. Real-World Patterns with TypeScript.
6. Practical Tips & Tricks.
7. Common Pitfalls to Avoid.
8. Adoption Strategy.

1.1. Just Enough Types: Stop Fighting Inference

// This case is handled in the code files section below

wrong.js

```
const userName: string = "Alice";  
const count: number = 42;  
const items: string[] = ["a", "b", "c"];
```

right.js

```
const userName = "Alice"; // TypeScript knows it's a string  
const count = 42;  
const items = ["a", "b", "c"];
```

Stop fighting inferences.

1.2. Just Enough Types: Type When It Matters

// This case is handled in the code files section below

typing.js

```
// Clear contract between caller and function
function formatUser(user: { name: string; age: number }): string {
  return `${user.name} (${user.age})`;
}

// API responses
interface ApiResponse {
  data: User[];
  error?: string;
}
```

Function boundaries ARE the right place.

1.3. Just Enough Types: The "Unknown" Safety Net

// This case is handled in the code files section below

any.js

```
function parseJSON(json: string): any {  
    return JSON.parse(json);  
}  
  
const data = parseJSON('{ "name": "Bob" }');  
data.doesNotExist.causesRuntimeError(); // No compile error!
```

unknown.js

```
function parseJSON(json: string): unknown {  
    return JSON.parse(json);  
}  
  
const data = parseJSON('{ "name": "Bob" }');  
  
// Must check before using  
if (typeof data === 'object' && data !== null && 'name' in data) {  
    console.log(data.name); // Safe!  
}
```

The "any" is an escape hatch.

Use "unknown" for safe type narrowing

Part 1: Summary

1. Just enough types: stop fighting inference.
2. Type when it matters.
3. The unknown safety net.

2.1. Bug Pattern Prevention: Discriminated Unions

// This case is handled in the code files section below

wrong.js

```
interface Response {
  success: boolean;
  data?: User;
  error?: string;
}

function handleResponse(res: Response) {
  if (res.success) {
    console.log(res.data.name); // Still might be undefined!
  }
}
```

right.js

```
type Response =
  | { status: 'success'; data: User }
  | { status: 'error'; error: string };

function handleResponse(res: Response) {
  if (res.status === 'success') {
    console.log(res.data.name); // TypeScript KNOWS data exists
  } else {
    console.log(res.error); // And here, error exists
  }
}
```

real.js

```
type LoadingState<T> =  
  | { status: 'idle' }  
  | { status: 'loading' }  
  | { status: 'success'; data: T }  
  | { status: 'error'; error: Error };  
  
function UserProfile({ state }: { state: LoadingState<User> }) {  
  switch (state.status) {  
    case 'idle':  
      return <div>Click to load</div>;  
    case 'loading':  
      return <Spinner />;  
    case 'success':  
      return <div>{state.data.name}</div>; // data guaranteed here  
    case 'error':  
      return <div>{state.error.message}</div>; // error guaranteed here  
  }  
}
```

Boolean flag nightmare that kills unknown checks.

Make invalid states impossible.

2.2. Literal Types (better than enums)

// This case is handled in the code files section below

wrong.js

```
function setTheme(theme: string) {  
  document.body.className = theme;  
}
```

```
setTheme("darkk"); // no error
```

right.js

```
type Theme = 'light' | 'dark' | 'auto';
```

```
function setTheme(theme: Theme) {  
  document.body.className = theme;  
}
```

```
setTheme("darkk"); // error
```

```
setTheme("dark"); // Autocomplete suggests options
```

real.js

```
type HttpMethod = 'GET' | 'POST' | 'PUT' | 'DELETE';
```

```
interface RequestConfig {  
  method: HttpMethod;  
  endpoint: `/api/${string}`; // Must start with /api/  
}
```

```
const config: RequestConfig = {  
  method: 'GET',  
  endpoint: '/api/users' //   
};
```

```
const bad: RequestConfig = {  
  method: 'GETS', // error  
  endpoint: 'api/users' // Error - missing leading slash  
};
```

Better than enums

Literal Types = autocomplete + safety

2.3. Readonly (stop accidental mutations)

// This case is handled in the code files section below

sneaky-bug.js

```
function processItems(items: string[]) {  
  items.sort(); // 🐛 Mutates the original array!  
  return items;  
}
```

```
const myList = ['c', 'a', 'b'];  
processItems(myList);  
console.log(myList); // ['a', 'b', 'c'] - Oops!
```

right.js

```
// Must create a copy to sort, can't mutate the original  
function processItems(items: readonly string[]): string[] {  
  return [...items].sort();  
}
```

```
// Or prevent mutation entirely
```

```
interface Config {  
  readonly apiKey: string;  
  readonly timeout: number;  
}
```

```
const config: Config = { apiKey: 'secret', timeout: 5000 };  
config.apiKey = 'new'; // ❌ Error - can't mutate
```

Readonly is a powerful tool for immutability.

Readonly arrays and objects prevent accidental mutations.

Readonly also makes intent clear to other developers.

Part 2: Summary

1. Discriminated Unions prevent bug patterns.
2. Literal Types are better than enums.
3. Readonly stops accidental mutations (makes intent clear).

3.1. Template Literal Types (type-safe routes)

// This case is handled in the code files section below

code.js

```
type Route = `/${users}/${string}` | `/${posts}/${string}` | '/home';

function navigate(route: Route) {
  window.location.href = route;
}

navigate('/users/123'); // ✓
navigate('/posts/abc'); // ✓
navigate('/user/123'); // ✗ Typo caught!
```

real.js

```
type CSSUnit = 'px' | 'em' | 'rem' | '%';
type Size = `${number}${CSSUnit}`;

function setWidth(width: Size) {
  element.style.width = width;
}

setWidth('100px'); // ✓
setWidth('50%'); // ✓
setWidth('100'); // ✗ Must include unit
```

testing.ts

```
type Route = `/${users}/${string}` | `/${posts}/${string}` | '/home';

function navigate(route: Route) {
  window.location.href = route;
}

navigate('/users/123'); // ✓
navigate('/posts/abc'); // ✓
navigate('/home'); // ✓
// navigate('/user/123'); // ✗ Typo caught!
```

Stop strongly-typed code.

Template literal types = safety + autocomplete for strings.

3.2. Index Signatures with Constraints (type-safe dictionaries)

// This case is handled in the code files section below

too-loose.js

```
interface UserPreferences {  
  [key: string]: any;  // Anything goes!  
}
```

constrained.js

```
interface UserPreferences {  
  [key: string]: string | number | boolean;  
}  
  
const prefs: UserPreferences = {  
  theme: 'dark',           //   
  fontSize: 14,            //   
  notifications: true,     //   
  callback: () => {}       //  Functions not allowed  
};
```

better.js

```
type Preferences = Record<string, string | number | boolean>;  
  
const prefs: Preferences = {  
  theme: 'dark',           //   
  fontSize: 14,            //   
  notifications: true,     //   
  callback: () => {}       //  Functions not allowed  
};
```

Type-safe dictionaries.

Index signatures with constraints = safety + autocomplete for object keys.

3.3. Satisfies Operator (type-safe objects)

// This case is handled in the code files section below

old-dilemma.js

```
// Option 1: Type annotation loses specificity
const config: Record<string, string> = {
  endpoint: 'https://api.example.com'
};

config['endpoint'].toUpperCase(); // Works, but...
const url = new URL(config['endpoint']); // Generic string, no URL specifics

// Option 2: No annotation loses safety
const config = {
  endpoint: 'https://api.example.com'
};

config.endpont; // ❌ Typo allowed!
```

satisfies.js

```
type Config = {
  endpoint: string;
  timeout: number;
};

const config = {
  endpoint: 'https://api.example.com',
  timeout: 5000,
} satisfies Config;

config.endpoint.toUpperCase(); // ✅ Knows it's a string
config.timeout.toFixed();      // ✅ Knows it's a number
config.endpont;                // ❌ Typo caught!
```

real.js

```
type RouteConfig = Record<string, { path: string; component: string }>;

const routes = {
  home: { path: '/', component: 'Home' },
  users: { path: '/users', component: 'UserList' },
  profile: { path: '/profile', component: 'Profile' }
} satisfies RouteConfig;

// Get autocomplete for route names:
const homePath = routes.home.path; // ✅ Autocomplete knows 'home' exists
const oopsPath = routes.hoem.path; // ❌ Typo caught!
```

testing.ts

```
// Option 1: Type annotation loses specificity
const config1: Record<string, string> = {
  endpoint: 'https://api.example.com'
};
config1['endpoint'].toUpperCase(); // Works, but...
const url = new URL(config1['endpoint']); // Generic string, no URL specifics

// Option 2: No annotation loses safety
const config2 = {
  endpoint: 'https://api.example.com'
};
config2.endpont; // ❌ Typo allowed!

type Config3 = {
  endpoint: string;
  timeout: number;
};

const config3 = {
  endpoint: 'https://api.example.com',
  timeout: 5000,
} satisfies Config3;

config3.endpoint.toUpperCase(); // ✅ Knows it's a string
config3.timeout.toFixed();      // ✅ Knows it's a number
config3.endpont;                // ❌ Typo caught!

type RouteConfig = Record<string, { path: string; component: string }>;

const routes = {
  home: { path: '/', component: 'Home' },
  users: { path: '/users', component: 'UserList' },
  profile: { path: '/profile', component: 'Profile' }
} satisfies RouteConfig;

// Get autocomplete for route names:
const homePath = routes.home.path; // ✅ Autocomplete knows 'home' exists
const oopsPath = routes.hoem.path; // ❌ Typo caught!
```

Type-safe objects.

Satisfies operator = safety + autocomplete for object values.

3.4. NonNullable and Utility Types (stop null/undefined bugs)

// This case is handled in the code files section below

code.js

```
type User = {
  id: string;
  email: string;
  nickname?: string;
};

// Extract only required fields
type RequiredUser = Required<User>;

// Extract only optional fields
type OptionalFields = {
  [K in keyof User as User[K] extends Required<User>[K] ? never : K]: User[K]
};

// Remove null/undefined
type Config = string | null | undefined;
type CleanConfig = NonNullable<Config>; // just string

// Real example - filtering arrays
const items = ['a', null, 'b', undefined, 'c'];
const clean = items.filter((x): x is NonNullable<typeof x> => x !== null);
// clean is string[], not (string | null | undefined)[]
```

testing.ts

```
type User = {
  id: string;
  email: string;
  nickname?: string;
};

// Extract only required fields
type RequiredUser = Required<User>;

// Extract only optional fields
type OptionalFields = {
  [K in keyof User as User[K] extends Required<User>[K] ? never : K]: User[K]
};

// Remove null/undefined
type Config = string | null | undefined;
type CleanConfig = NonNullable<Config>; // just string

// Real example - filtering arrays
const items = ['a', null, 'b', undefined, 'c'];
const clean = items.filter((x): x is NonNullable<typeof x> => x !== null);
// clean is string[], not (string | null | undefined)[]

console.log(clean); // ['a', 'b', 'c'];
```

Stop null/undefined bugs.

NonNullable and utility types = safety + intent clarity for null/undefined.

Part 3: Summary

1. Template Literal Types for type-safe strings.
2. Index Signatures with Constraints for type-safe dictionaries.
3. Satisfies Operator for type-safe objects.
4. NonNullable and Utility Types to stop null/undefined bugs.

4.1. DRY Derive Types with "Omit," "Partial," and "Pick"

// This case is handled in the code files section below

wrong.js

```
interface CreateUserRequest {
  name: string;
  email: string;
  age: number;
}

interface UpdateUserRequest {
  name: string;      // Duplicated!
  email: string;     // Duplicated!
  age: number;       // Duplicated!
}
```

right.js

```
interface User {
  id: string;
  name: string;
  email: string;
  age: number;
  createdAt: Date;
}

type CreateUserRequest = Omit<User, 'id' | 'createdAt'>;
type UpdateUserRequest = Partial<CreateUserRequest>;
type UserResponse = Pick<User, 'id' | 'name' | 'email'>;
```

DRY Derive Types with "Omit," "Partial," and "Pick".

Use "Omit," "Partial," and "Pick" to avoid repetition and keep types in sync.

4.2. Let Code Generate Types ("typeof" and "keyof")

// This case is handled in the code files section below

correct.js

```
// Define data structure once
const API_ENDPOINTS = {
  users: '/api/users',
  posts: '/api/posts',
  comments: '/api/comments'
} as const;

// Generate types from it
type Endpoint = keyof typeof API_ENDPOINTS;
type EndpointPath = typeof API_ENDPOINTS[Endpoint];

function fetchData(endpoint: Endpoint) {
  return fetch(API_ENDPOINTS[endpoint]);
}

fetchData('users');    // ✅ Autocomplete!
fetchData('userss');   // ❌ Error
```

real.js

```
const eventHandlers = {
  click: (e: MouseEvent) => {},
  keypress: (e: KeyboardEvent) => {},
  scroll: (e: UIEvent) => {}
};

type EventType = keyof typeof eventHandlers;
type EventHandler<T extends EventType> = typeof eventHandlers[T];

function addEventListener<T extends EventType>(
  type: T,
  handler: EventHandler<T>
) {
  // Implementation
}
```

Let code generate types with "typeof" and "keyof".

Use "typeof" and "keyof" to create types based on existing values and keys.

Part 4: Summary

1. DRY Derive Types with Omit, Partial, and Pick.
2. Let Code Generate Types with typeof and keyof.

5.1. Type-Safe Event Emitters

// This case is handled in the code files section below

wrong.js

```
emitter.on('user:created', (data) => {  
  console.log(data.name); // What properties exist?  
});
```

```
emitter.emit('user:crated', user); // Typo!
```

right.js

```
interface Events {  
  'user:created': { id: string; name: string };  
  'user:deleted': { id: string };  
  'post:published': { postId: string; title: string };  
}  
  
class TypedEmitter {  
  on<K extends keyof Events>(  
    event: K,  
    handler: (data: Events[K]) => void  
  ): void {  
    // Implementation  
  }  
  
  emit<K extends keyof Events>(event: K, data: Events[K]): void {  
    // Implementation  
  }  
}  
  
const emitter = new TypedEmitter();  
  
emitter.on('user:created', (data) => {  
  console.log(data.name); // ✅ Autocomplete!  
});  
  
emitter.emit('user:crated', { id: '1', name: 'Alice' }); // ❌ Typo caught!  
emitter.emit('user:created', { id: '1' }); // ❌ Missing 'name'
```

Type-safe event emitters.

Use generics and mapped types to create type-safe event emitters.

5.2. Builder Pattern with Type State

// This case is handled in the code files section below

code.js

```
interface ServerConfig {
  host: string;
  port: number;
  apiKey: string;
}

// Track which fields have been set
type ConfigBuilder<T extends Partial<ServerConfig> = {}> = {
  setHost: (host: string) => ConfigBuilder<T & { host: string }>;
  setPort: (port: number) => ConfigBuilder<T & { port: number }>;
  setApiKey: (key: string) => ConfigBuilder<T & { apiKey: string }>;
  build: T extends ServerConfig ? () => ServerConfig : never;
};

function createConfigBuilder<T extends Partial<ServerConfig> = {}>(state: T = {} as T):
ConfigBuilder<T> {
  return {
    setHost: (host: string) => createConfigBuilder({ ...state, host }),
    setPort: (port: number) => createConfigBuilder({ ...state, port }),
    setApiKey: (key: string) => createConfigBuilder({ ...state, apiKey: key }),
    build: (() => state) as any,
  };
}

// Usage
const config = createConfigBuilder()
  .setHost('localhost')
  .setPort(3000)
  .setApiKey('secret')
  .build(); // ✅ Only works when all fields set

const incomplete = createConfigBuilder()
  .setHost('localhost')
  .build(); // ❌ Error - missing required fields
```

testing.ts

```
interface ServerConfig {
  host: string;
  port: number;
  apiKey: string;
}

// Track which fields have been set
type ConfigBuilder<T extends Partial<ServerConfig> = {}> = {
  setHost: (host: string) => ConfigBuilder<T & { host: string }>;
  setPort: (port: number) => ConfigBuilder<T & { port: number }>;
  setApiKey: (key: string) => ConfigBuilder<T & { apiKey: string }>;
  build: T extends ServerConfig ? () => ServerConfig : never;
};

function createConfigBuilder<T extends Partial<ServerConfig> = {}>(state: T = {} as T):
ConfigBuilder<T> {
  return {
    setHost: (host: string) => createConfigBuilder({ ...state, host }),
    setPort: (port: number) => createConfigBuilder({ ...state, port }),
    setApiKey: (key: string) => createConfigBuilder({ ...state, apiKey: key }),
    build: () => state as any,
  };
}

// Usage
const config = createConfigBuilder()
  .setHost('localhost')
  .setPort(3000)
  .setApiKey('secret')
  .build(); // ✅ Only works when all fields set

const incomplete = createConfigBuilder()
  .setHost('localhost')
  .build(); // ❌ Error - missing required fields
```

Builder pattern with type state.

Use generics and conditional types to create a builder pattern that enforces correct method chaining.

5.3. Type-Safe Dependency Injection

// This case is handled in the code files section below

code.js

```
// Define service interfaces
interface Logger {
  log(message: string): void;
}

interface Database {
  query<T>(sql: string): Promise<T>;
}

interface Services {
  logger: Logger;
  database: Database;
}

// Container with type safety
class Container {
  private services = new Map<keyof Services, any>();

  register<K extends keyof Services>(
    name: K,
    service: Services[K]
  ): void {
    this.services.set(name, service);
  }

  get<K extends keyof Services>(name: K): Services[K] {
    return this.services.get(name);
  }
}

// Usage
const container = new Container();
container.register('logger', consoleLogger);
container.register('databse', db); // ❌ Typo caught!

const logger = container.get('logger');
logger.log('Hello'); // ✅ Fully typed
```

testing.ts

```
// Define service interfaces
interface Logger {
  log(message: string): void;
}

interface Database {
  query<T>(sql: string): Promise<T>;
}

interface Services {
  logger: Logger;
  database: Database;
}

// Container with type safety
class Container {
  private services = new Map<keyof Services, any>();

  register<K extends keyof Services>(
    name: K,
    service: Services[K]
  ): void {
    this.services.set(name, service);
  }

  get<K extends keyof Services>(name: K): Services[K] {
    return this.services.get(name);
  }
}

// Usage
const consoleLogger: Logger = {
  log(message: string): void {
    console.log(message);
  }
};

const container = new Container();
container.register('logger', consoleLogger);
container.register('databse', db); // ❌ Typo caught!

const logger = container.get('logger');
logger.log('Hello'); // ✅ Fully typed
```

Type-safe dependency injection.

Use interfaces and generics to create a type-safe dependency injection system.

Part 5: Summary

1. Type-Safe Event Emitters.
2. Builder Pattern with Type State.
3. Type-Safe Dependency Injection.

6.1. Exhaustive Switch Checks

// This case is handled in the code files section below

code.js

```
type Status = 'pending' | 'approved' | 'rejected';

function handleStatus(status: Status) {
  switch (status) {
    case 'pending':
      return 'Waiting...';
    case 'approved':
      return 'Success!';
    case 'rejected':
      return 'Failed';
    default:
      // This function ensures we handled all cases
      const _exhaustive: never = status;
      throw new Error(`Unhandled status: ${_exhaustive}`);
  }
}

// Later, someone adds a new status
type Status = 'pending' | 'approved' | 'rejected' | 'cancelled';

// The switch now shows an error at compile time! ✅
```

Exhaustive switch checks.

Use discriminated unions and the never type to ensure all cases are handled in switch statements.

6.2. Branded Types (Prevent Mixing Similar Types)

// This case is handled in the code files section below

wrong.js

```
// Problem: Both are just strings
type UserId = string;
type PostId = string;

function getUser(id: UserId) {}
function getPost(id: PostId) {}

const userId: UserId = '123';
const postId: PostId = '456';

getUser(postId); // ❌ Should error, but doesn't!
```

right.js

```
type UserId = string & { readonly __brand: 'UserId' };
type PostId = string & { readonly __brand: 'PostId' };

function createUserId(id: string): UserId {
  return id as UserId;
}

function createPostId(id: string): PostId {
  return id as PostId;
}

function getUser(id: UserId) {}
function getPost(id: PostId) {}

const userId = createUserId('123');
const postId = createPostId('456');

getUser(postId); // ❌ Type error!
getUser(userId); // ✅ Works
```

Branded types.

Use branded types to create nominal types for added type safety.

6.3. Assert Functions

// This case is handled in the code files section below

code.js

```
// Better than throwing in conditionals
function assertIsDefined<T>(  
  value: T,  
  message?: string  
) : asserts value is NonNullable<T> {  
  if (value === undefined || value === null) {  
    throw new Error(message ?? 'Value is null or undefined');  
  }  
}  
  
function processUser(user: User | null) {  
  assertIsDefined(user, 'User must be defined');  
  // TypeScript now knows user is NOT null  
  console.log(user.name); // ✅ No optional chaining needed  
}
```

Assert functions (custom type guards).

Use assert functions to create custom type guards for complex type checks.

6.4. Type Predicates for Array Filtering

// This case is handled in the code files section below

code.js

```
const values = ['a', null, 'b', undefined, 'c', null];

// ❌ TypeScript doesn't narrow the type
const filtered = values.filter(x => x !== null);
// filtered is still (string | null | undefined)[]

// ✅ Use type predicate
function isNotNull<T>(value: T | null | undefined): value is T {
    return value !== null;
}

const filtered = values.filter(isNotNull);
// filtered is now string[] ✅
```

Type predicates for array filtering.

Use type predicates to create type-safe array filters that narrow types.

Part 6: Summary

1. Exhaustive Switch Checks with discriminated unions and the never type.
2. Branded Types to prevent mixing similar types.
3. Assert Functions (custom type guards) for complex type checks.
4. Type Predicates for Array Filtering to create type-safe array filters.

7.1. Do NOT Abuse Generics

// This case is handled in the code files section below

wrong.js

```
// Over Engineering
function getValue<T, K extends keyof T, V extends T[K]>(
  obj: T,
  key: K
): V {
  return obj[key] as V;
}
```

right.js

```
function getValue<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}
```

Generics are powerful, but they can be overused.

Don't use generics when a specific type is sufficient.

7.2. Avoid Type Assertions (unless necessary)

// This case is handled in the code files section below

wrong.js

```
// Lying to the compiler
// ... Bomb waiting to explode
const data = apiResponse as User;
```

right.js

```
// Validate instead of asserting
function isUser(data: unknown): data is User {
  return (
    typeof data === 'object' &&
    data !== null &&
    'id' in data &&
    'email' in data
  );
}

if (isUser(apiResponse)) {
  console.log(apiResponse.email); // ✅ Safe
}
```

Type assertions ("as") can be a sign of a design problem.

Avoid type assertions by improving your types or using "unknown" for safe narrowing.

7.3. Do NOT Fight the Compiler

// This case is handled in the code files section below

code.js

```
// The ANY escape hatch
function hackyFunction(data: any) { // Giving up
    return data.whatever.you.want;
}

// Ask "why is this hard?"
// * Often means your types don't match reality
// * Refactor the types, not add "any"
```

Don't fight the compiler.

Embrace TypeScript's type system and let it guide your code design for better safety and maintainability.

Part 7: Summary

1. Do NOT Abuse Generics.
2. Avoid Type Assertions (unless necessary).
3. Do NOT Fight the Compiler.

8.1. Start Small: Incremental Adoption

// This case is handled in the code files section below

code.js

```
// 1. Enable TypeScript, but allow JS

// tsconfig.json
{
  "compilerOptions": {
    "allowJs": true,
    "checkJs": false,  // Don't check JS files yet
    "strict": false    // Start loose
  }
}

// 2. Convert one file at a time (.js → .ts)
// 3. Gradually enable strict flags
{
  "strict": true,  // or enable individually:
  "strictNullChecks": true,
  "strictFunctionTypes": true
}
```

Start small with incremental adoption.

Introduce TypeScript gradually to your codebase, starting with new files or modules.

8.2. The 80/20 Rule

Focus on high-value areas first:

1. API boundaries (requests/responses).
2. Shared utility functions.
3. State management.
4. Configuration objects.

Skip (for now):

- One-off scripts.
- Legacy code you rarely touch.
- Over-typed utility functions.

The 80/20 rule: focus on high-impact areas.

Focus on high-value areas first:

1. API boundaries (requests/responses)
2. Shared utility functions
3. State management
4. Configuration objects

Skip (for now):

- One-off scripts
- Legacy code you rarely touch
- Over-typed utility functions

Part 8: Summary

1. Start small with incremental adoption.
2. Focus on high-value areas first (80/20 rule).

Quick Wins You Can Apply Today

// This case is handled in the code files section below

code.js

```
// ✅ Literal types for constants
type HttpMethod = 'GET' | 'POST' | 'PUT' | 'DELETE';

// ✅ Readonly for immutability
function process(data: readonly string[]) {}

// ✅ satisfies for safe configuration
const config = { /*...*/ } satisfies ConfigShape;

// ✅ Type predicates for filtering
const clean = items.filter((x): x is string => typeof x === 'string');

// ✅ Exhaustive checks
default: const _: never = value;
```

- Literal types for constants
- Readonly for immutability
- satisfies for safe configuration
- Type predicates for filtering
- Exhaustive checks

Key Takeaways

1. Type at boundaries, infer in the middle.
2. Make invalid states unrepresentable (discriminated unions).
3. Derive types from single sources (DRY principle).
4. Use unknown, not any.
5. TypeScript is a tool, not a test of purity.

1. Type at boundaries, infer in the middle.
2. Make invalid states unrepresentable (discriminated unions).
3. Derive types from single sources (DRY principle).
4. Use unknown, not any.
5. TypeScript is a tool, not a test of purity.

Resources for Further Learning

Resources

- Official TypeScript Documentation: <https://www.typescriptlang.org/docs>
- TypeScript Playground: <https://www.typescriptlang.org/play>
- TypeScript Community: <https://www.typescriptlang.org/community>
- Total TypeScript: <https://totaltypescript.com> (Matt Pocock)
- Type Challenges: <https://github.com/type-challenges/type-challenges>

The Philosophy

"Perfect types don't exist. Better types do."

"Start where you are. Use what helps. Skip what doesn't."

- TypeScript doesn't have to be painful.
- These patterns have saved hours of debugging, prevented production bugs, and made the code easier to refactor.
- Pick one or two that resonate with you, try them, and see if TypeScript becomes something you MIGHT actually ... like.

